VIT®

LABORATORY MANUAL

COMPUTATIONAL ENGINEERING

Course Code: BAMEE301



SCHOOL OF MECHANICAL ENGINEERING

VIT CHENNAI

COMPUTATIONAL ENGINEERING

LABORATORY MANUAL

Course Code: BAMEE301, Fall Semester 2026-27

Reg. No.	
Name	
Semester	
Class No.	
Slot	

Exp. No.	Name of the Experiments	Date Conducted	Date Verified	Faculty Sign
1	Write a program to compute a given Transcendental function using Taylor series for different step sizes and analyze round-off and truncation errors.			
2	Implement Gaussian elimination and LU decomposition to solve a given system of equations and compare computational efficiency.			
3	Write programs to implement Jacobi and Gauss–Seidel methods for solving a system of linear equations, and study convergence with different relaxation factors.			
4	Write programs to implement and compare the bisection, Newton–Raphson, and secant methods for solving an appropriate mechanical engineering problem.			
5	A simply supported beam is instrumented at midspan to measure transverse displacement under harmonic forcing. The measured displacement (in meters) is: $$u(t) = 0.005\sin(20t) + 0.002\cos(15t)$$ Determine the midspan velocity and acceleration of a simply supported beam subjected to harmonic loading using the given discrete displacement–time data. Employ forward, backward, and central difference schemes to numerically approximate the velocity and acceleration. Compare the obtained results with the corresponding analytical derivatives, and analyze the accuracy, stability, and suitability of each numerical scheme in relation to modal response, sensor sampling, and experimental noise in mechanical engineering practice.			

6	A piston in a cylinder is subjected to a damped oscillatory force during its motion. The variation of the applied force $F(x)$ (in newtons) with piston displacement x (in meters) is described by $$F(x) = 500e^{-5x}\, sin(10x)$$ The work done by the force as the piston moves from $x_1 = 0\ m$ to $x_2 = 0.5\ m$ is given by $$W = \int_{x_1}^{x_2} F(x)\ dx$$ Write programs to evaluate the work done using trapezoidal, Simpson's and Gaussian quadrature methods.			
7	Solve an initial-value problem for an ODE that resembles the velocity response of a damped system under harmonic excitation using Euler's method, $$\frac{dy}{dt} = -2y + cos(t),\ \ y(0) = 1$$ Compare the numerical solution with the analytical solution and study the effect of step size on accuracy.			
8	Solve two-dimensional steady state heat conduction equation given by, $$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0$$ for a rectangular domain with specified boundary conditions using central differences.			
9	Write a program to numerically solve and analyze the velocity evolution of an incompressible, Newtonian fluid in an instantaneously started Couette flow between two parallel plates using explicit finite difference method. Compare the numerically obtained transient velocity profile with the analytical solution. The following equation governs this physical flow scenario. $$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial y^2}$$ Use appropriate boundary conditions.			
10	Solve the flowing wave equation applicable for the acoustic signals $$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$$ using the explicit central-difference scheme and study numerical stability.			

Exp. No :	**Write a program to compute a given Transcendental function using Taylor series for different step sizes and analyze round-off and truncation errors.**	Date :

AIM:

To compute a transcendental function using Taylor series expansion for different step sizes and to analyze the truncation and round-off errors associated with the numerical computation.

OBJECTIVES:

1. To understand the Taylor series representation of transcendental functions.
2. To evaluate a transcendental function numerically using a finite number of Taylor series terms.
3. To study the effect of step size on numerical computations.
4. To determine truncation and round-off errors.
5. To compare numerical results with exact analytical solutions.
6. Plot the results

THEORY :

In engineering practice, many physical phenomena are represented by transcendental functions such as exponential, trigonometric, and logarithmic functions. Examples include vibration response of structures, heat transfer in solids, fluid flow, damping characteristics, and control system dynamics. Since these functions often cannot be evaluated analytically during numerical computations, they are approximated using series expansions such as the Taylor Series.

The Taylor Series provides a polynomial approximation of a function about a specified point and forms the foundation of many numerical methods used in Computational Engineering.

Taylor Series Expansion

A function $f(x)$ can be expanded about $x = 0$ (Maclaurin Series) as:

$$f(x) = f(0) + xf'(0) + \frac{x^2}{2!}f''(0) + \frac{x^3}{3!}f'''(0) + \cdots$$

For the exponential function:

$$e^x \approx 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!}$$

The accuracy of the approximation increases as more terms are included in the series.

NUMERICAL COMPUTATION AND STEP SIZE

CONCEPT

In numerical methods, a continuous function is evaluated at discrete points separated by a fixed interval called the **step size (h)**.

DISCRETE POINT GENERATION

$$x_i = x_0 + ih$$

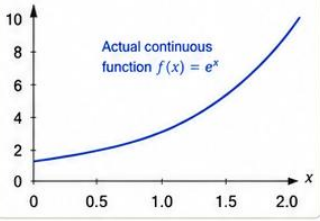
x_0 : starting point

h : step size

i : 0, 1, 2, 3, . . .

EXAMPLE FUNCTION

$f(x)$ $f(x) = e^x$ over $0 \le x \le 2$



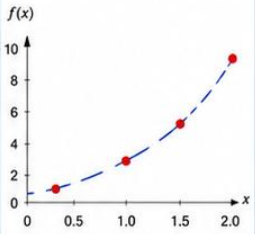
EFFECT OF STEP SIZE

Step Size (h)	Number of Points (N)	Accuracy	Computational Cost
0.50	5	Low	Very Low
0.25	9	Moderate	Low
0.10	21	High	Moderate
0.05	41	Very High	High

- **Larger h** → faster computation but lower accuracy
- **Smaller h** → higher accuracy but greater computational effort

1. LARGE STEP SIZE ($h = 0.5$)

$x = 0, 0.5, 1.0, 1.5, 2.0$

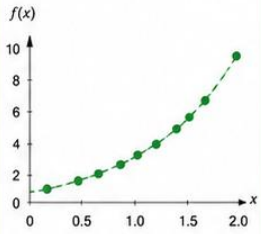


Number of points $N = \frac{2-0}{0.5} + 1 = 5$

+ Less computation time
− Lower accuracy

2. MEDIUM STEP SIZE ($h = 0.25$)

$x = 0, 0.25, 0.5, 0.75, ..., 2.0$

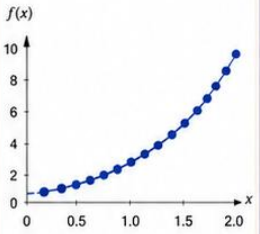


Number of points $N = \frac{2-0}{0.25} + 1 = 9$

+ Better accuracy
− More computation

3. SMALL STEP SIZE ($h = 0.10$)

$x = 0, 0.1, 0.2, ..., 2.0$



Number of points $N = \frac{2-0}{0.10} + 1 = 21$

+ High accuracy
− High computational cost

MECHANICAL ENGINEERING EXAMPLE

Vibration of a machine foundation:

$$u(t) = 0.005 \sin(20t)$$

Large Time Step ($\Delta t = 0.1$ s)



Few samples → Peaks may be missed

Small Time Step ($\Delta t = 0.01$ s)



More samples → Accurate capture of vibration waveform

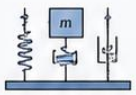
EFFECT OF STEP SIZE ON ACCURACY

Step Size (h)	Number of Points (N)	Advantages	Disadvantages
0.50	5	• Less computation time • Lower memory requirement	• Lower accuracy • Poor representation of rapidly varying functions
0.25	9	• Improved accuracy • Moderate computation time	• May not capture sharp variations accurately
0.10	21	• Better approximation • Good balance of accuracy and cost	• Higher computation time
0.05	41	• Very high accuracy	• Very high computational cost

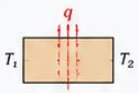
KEY OBSERVATION

- **Larger step size** → faster computation but lower accuracy.
- **Smaller step size** → higher accuracy but greater computational effort.
- An optimum step size must be selected to balance **accuracy**, **stability** and **computational efficiency**.

APPLICATIONS IN MECHANICAL ENGINEERING



Vibration Analysis and Dynamics



Heat Transfer Problems



Fluid Flow and CFD Simulations



Finite Difference Methods



Structural Analysis and FEM



Signal Processing and Control

Note: Selection of step size is one of the most important decisions in numerical computation as it directly influences the accuracy, stability and efficiency of the numerical solution.

Develop an algorithm and MATLAB program to compute any two of the following transcendental functions using Taylor series approximation. Study the effect of different step

sizes / Taylor terms on numerical computation, plot the results, and determine the truncation and round-off errors.

Sl. No	Details	Function	Interval	Taylor Series Terms / Steps (N) / Step Size (h)
1	Damped mechanical system	$f(x) = e^{-x}$	$0 \leq x \leq 2$	3, 5, 7 and 10
2	Harmonic Vibration Response of a Machine Foundation	$u(t) = 0.005 \sin(20t)$	$0 \leq t \leq 1\ s$	0.1, 0.05, 0.01
3	Transient Cooling of a Heated Component	$T(t) = 100e^{-0.5t}$	$0 \leq t \leq 5\ s$	3, 5, 7 and 10
4	Stress Relaxation in a Polymer Component	$\sigma(t)=200e^{-0.1t}$	$0 \leq t \leq 10\ s$	0.1, 0.05, 0.01
5	Damped Vibration Response of a Mechanical System	$x(t)=0.01e^{-0.2t}\sin(10t)$	$0 \leq t \leq 3s$	0.1, 0.05, 0.01

ALGORITHM / PROCEDURE:

RESULT TABLE WITH ROUND OFF AND TRUNCATION ERRORS

MATLAB output plot (Please paste the plot here)

EVALUATION RUBRICS	MAX. MARKS	ACTUAL MARKS
Algorithm Procedures	4	
MATLAB Program and execution	3	
Plots / Output	2	
Understanding	1	
Total	10	
Faculty Signature with Date:		

Exp. No : 2	**Implement Gaussian elimination and LU decomposition to solve a given system of equations and compare computational efficiency.**	Date :

AIM:

To implement Gaussian elimination and LU decomposition methods for solving a system of simultaneous linear equations and to compare their accuracy and computational efficiency.

Objectives

1. To represent an engineering system in matrix form:

$$[A]\{x\} = \{b\}$$

2. To solve the system using Gaussian elimination with partial pivoting.
3. To decompose the coefficient matrix into lower- and upper-triangular matrices using LU decomposition:

$$[A] = [L][U]$$

4. To determine the unknown variables by forward and backward substitution.
5. To verify the computed solution using the residual vector:

$$\{R\} = [A]\{x\} - \{b\}$$

6. To compare Gaussian elimination and LU decomposition in terms of computational time, arithmetic operations, accuracy, and suitability for multiple right-hand-side vectors.

Problem Statement :

A four-degree-of-freedom spring system is represented by the following system of simultaneous linear equations:

$$[K]\{x\} = \{F\}$$

where the stiffness matrix is

$$[K] = \begin{bmatrix} 4 & -1 & 0 & 0 \\ -1 & 4 & -1 & 0 \\ 0 & -1 & 4 & -1 \\ 0 & 0 & -1 & 3 \end{bmatrix}$$

and $\{x\}$represents the unknown displacements.

Load Case 1

$$\{F_1\} = \begin{bmatrix} 2 \\ 4 \\ 6 \\ 9 \end{bmatrix}$$

Load Case 2

$$\{F_2\} = \begin{bmatrix} 7 \\ 2 \\ -4 \\ 9 \end{bmatrix}$$

Theory

Many engineering problems lead to a system of simultaneous linear equations. Examples include structural deformation, heat transfer, electrical networks, fluid-flow discretization, and finite-element analysis. The system can be written in matrix form as

$$[A]\{x\} = \{b\}$$

where:

- $[A]$is the coefficient matrix,
- $\{x\}$is the vector of unknowns,
- $\{b\}$is the known right-hand-side vector.

1. Gaussian Elimination

Gaussian elimination transforms the original system into an equivalent upper-triangular system.

$$[A]\{x\} = \{b\}$$

is reduced to

$$[U]\{x\} = \{c\}$$

where $[U]$is upper triangular.

For a three-equation system,

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 &= b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 &= b_2 \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3 \end{aligned}$$

the elements below the main diagonal are eliminated using row operations.

The multiplier used to eliminate $a_{ik}$is

$$m_{ik} = \frac{a_{ik}}{a_{kk}}$$

The row is updated as

$$R_i \leftarrow R_i - m_{ik} R_k$$

After obtaining the upper-triangular matrix, the unknowns are calculated by backward substitution.

$$x_n = \frac{c_n}{u_{nn}}$$

and

$$x_i = \frac{c_i - \sum_{j=i+1}^{n} u_{ij}\, x_j}{u_{ii}}$$

Partial Pivoting

If a pivot element is zero or very small, numerical instability may occur. In partial pivoting, the current row is interchanged with another row having the largest absolute value in the pivot column.

This reduces division by small numbers and improves numerical stability.

2. LU Decomposition

In LU decomposition, the coefficient matrix is factored as

$$[A] = [L][U]$$

where:

- $[L]$is a lower-triangular matrix,
- $[U]$is an upper-triangular matrix.

The original equation becomes

$$[L][U]\{x\} = \{b\}$$

Introduce an intermediate vector $\{y\}$:

$$[L]\{y\} = \{b\}$$

This equation is solved using forward substitution. Then,

$$[U]\{x\} = \{y\}$$

is solved using backward substitution. For forward substitution,

$$y_i = \frac{b_i - \sum_{j=1}^{i-1} l_{ij}\, y_j}{l_{ii}}$$

For backward substitution,

$$x_i = \frac{y_i - \sum_{j=i+1}^{n} u_{ij}\, x_j}{u_{ii}}$$

Computational Efficiency

For a single right-hand-side vector, Gaussian elimination and LU decomposition require similar computational effort. However, when the same coefficient matrix is used with several load vectors,

$$[A]\{x_1\} = \{b_1\}, [A]\{x_2\} = \{b_2\}$$

LU decomposition is more efficient because $[A]$is decomposed only once. The same matrices $[L]$and $[U]$are reused for every new right-hand-side vector. Only forward and backward substitution must be repeated. In Gaussian elimination, the elimination process is generally repeated for each load case unless the eliminated form is stored and reused.

Accuracy Verification

The computed solution is checked using the residual vector:

$$\{R\} = [A]\{x\} - \{b\}$$

For an accurate solution,

$$\{R\} \approx \{0\}$$

The residual norm may be calculated as

$$\| R \|_2 = \sqrt{\sum_{i=1}^{n} R_i^2}$$

A smaller residual norm indicates better numerical agreement with the original equations.

ALGORITHM / PROCEDURE:

Required Output

Displacements	Value	
	Gaussian Elimination Results	**LU Decomposition Results**
x_1		
x_2		
x_3		
x_4		

Method	Load Case	Computed Displacements	Residual Norm	Computational Time
Gaussian elimination	1			
Gaussian elimination	2			
LU decomposition	1			
LU decomposition	2			

Inference :

EVALUATION RUBRICS	MAX. MARKS	ACTUAL MARKS
Algorithm Procedures	4	
MATLAB Program and execution	3	
Plots / Output	2	
Understanding	1	
Total	10	
Faculty Signature with Date:		

Exp. No : 3	Write programs to implement Jacobi methods for solving a system of linear equations, and study convergence with different relaxation factors.	Date :

1. Aim

To develop a MATLAB program to solve a system of simultaneous linear equations using the Jacobi Iterative Method and study its convergence.

2. Objectives

After completing the experiment, the student should be able to:

- Represent an engineering problem in the matrix form .
- Check the diagonal dominance of the coefficient matrix.
- Implement the Jacobi iterative algorithm in MATLAB.
- Apply a convergence tolerance and determine the number of iterations.
- Evaluate the residual error of the computed solution.
- Plot convergence error versus iteration number.
- Verify the iterative solution using MATLAB's direct solver

3. Theory

Jacobi Method – Mathematical Formulation

A system of n simultaneous linear equations can be written as

$$[A]\{x\} = \{b\}$$

where

$[A]$ = coefficient matrix

$\{x\}$ = unknown vector

$\{b\}$ = known right-hand-side vector

For example,

$$a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1$$

and similarly for the remaining equations.

In the **Jacobi method**, every unknown is rearranged in terms of the remaining unknowns.

For a three-variable system,

$$x_1^{(k+1)} = \frac{1}{a_{11}}\left[b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)}\right]$$
$$x_2^{(k+1)} = \frac{1}{a_{22}}\left[b_2 - a_{21}x_1^{(k)} - a_{23}x_3^{(k)}\right]$$
$$x_3^{(k+1)} = \frac{1}{a_{33}}\left[b_3 - a_{31}x_1^{(k)} - a_{32}x_2^{(k)}\right]$$

The important feature of the Jacobi method is:

All new values are calculated using only the previous iteration values.

4. Convergence Criterion

Convergence Criterion

The iterations are continued until

$$\max_i \left| x_i^{(k+1)} - x_i^{(k)} \right| < \varepsilon$$

For this experiment, use

$$\varepsilon = 10^{-5}$$

5. Diagonal Dominance

Before applying the Jacobi method, check whether the matrix is **diagonally dominant**.

For every row,

$$|a_{ii}| \geq \sum_{j \neq i} |a_{ij}|$$

Strict diagonal dominance generally **ensures convergence** of the Jacobi method.

6. Engineering Problem

A steady-state heat-transfer model of four interconnected thermal nodes gives the equations

$$10T_1 - T_2 - 2T_3 = 72$$
$$-2T_1 + 12T_2 - T_3 - T_4 = 85$$
$$-T_1 - 2T_2 + 10T_3 - T_4 = 64$$
$$-T_2 - 2T_3 + 8T_4 = 51$$

where T_1, T_2, T_3, T_4 are the unknown nodal temperatures.

In matrix form,

$$\begin{bmatrix} 10 & -1 & -2 & 0 \\ -2 & 12 & -1 & -1 \\ -1 & -2 & 10 & -1 \\ 0 & -1 & -2 & 8 \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} = \begin{bmatrix} 72 \\ 85 \\ 64 \\ 51 \end{bmatrix}$$

Use the initial approximation

$$T_1^{(0)} = T_2^{(0)} = T_3^{(0)} = T_4^{(0)} = 0$$

7. Jacobi Iteration Equations

Jacobi Iteration Equations

From the first equation,

$$T_1^{(k+1)} = \frac{72 + T_2^{(k)} + 2T_3^{(k)}}{10}$$

From the second equation,

$$T_2^{(k+1)} = \frac{85 + 2T_1^{(k)} + T_3^{(k)} + T_4^{(k)}}{12}$$

From the third equation,

$$T_3^{(k+1)} = \frac{64 + T_1^{(k)} + 2T_2^{(k)} + T_4^{(k)}}{10}$$

From the fourth equation,

$$T_4^{(k+1)} = \frac{51 + T_2^{(k)} + 2T_3^{(k)}}{8}$$

Sample Output Table (Modify according to the Matrix size)

Iteration	(T_1)	(T_2)	(T_3)	(T_4)	Maximum Error
0	0	0	0	0	—
1					
...					
Final					

Plot (Maximum Absolute error vs Iteration number)

Inference :

EVALUATION RUBRICS	MAX. MARKS	ACTUAL MARKS
Algorithm Procedures	4	
MATLAB Program and execution	3	
Plots / Output	2	
Understanding	1	
Total	10	
Faculty Signature with Date:		

